

99 - Create custom authorization policy using func

Why Create a Custom Policy

➤ Create a policy with multiple requirements

- Must be a member of the **Admin** role AND
- Must have **Edit Role** claim with a value of **true** OR
- Must be a member of the **Super Admin** role

```
options.AddPolicy("EditRolePolicy",  
    policy => policy.RequireClaim("Edit Role", "true"));  
options.AddPolicy( (parameter) AuthorizationPolicyBuilder policy
```

Както се вижда от Intellisense, параметър policy е от тип AuthorizationPolicyBuilder. Този обект поддържа fluence index, което означава, че не поддържа множество правила и роли наведнъж.

```
services.AddAuthorization(options =>  
{  
    options.AddPolicy("EditRolePolicy", policy => policy  
        .RequireRole("Admin")  
        .RequireClaim("Edit Role", "true")  
        .RequireRole("Super Admin")  
    });  
});
```



Обяснението е много просто: при подобен запис, всички 3 изисквания трябва да бъдат изпълнени. На нас това не ни върши работа, макар синтаксисът да е вярно изписан. Ние искаме първите 2 условия да бъдат изпълнение ИЛИ 3-тото.

За да го докажем, правим демонстрация:

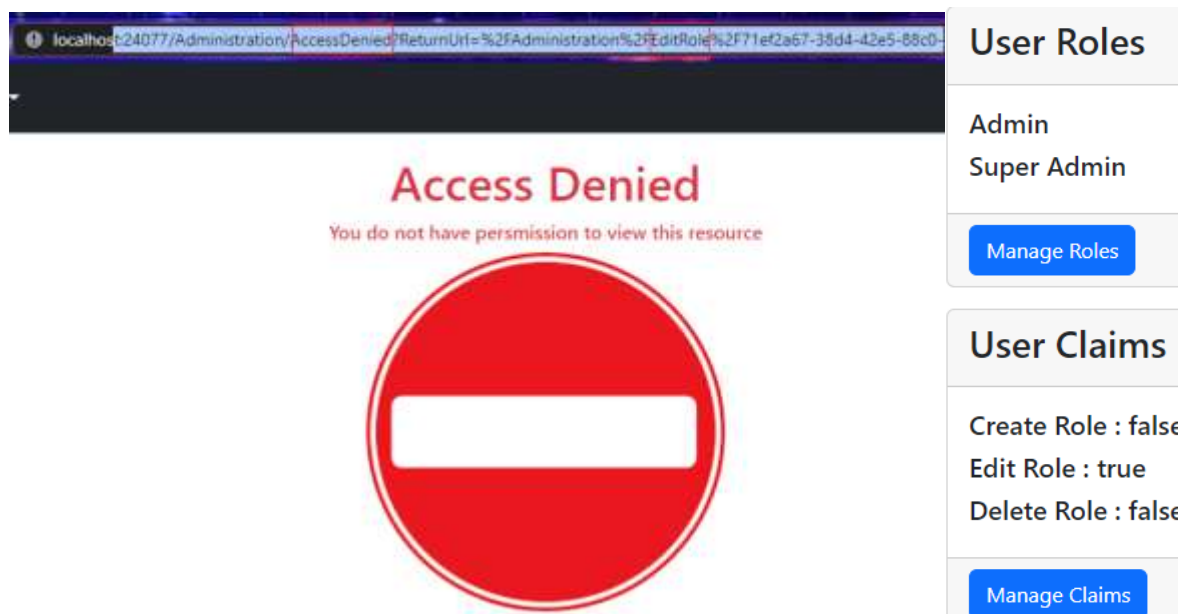
```
options.AddPolicy("EditRolePolicy",  
    policy => policy.RequireClaim("Edit Role", "true")  
        .RequireRole("Admin")  
        .RequireRole("Super Admin"));
```

```
[HttpGet]  
[Authorize(Policy = "EditRolePolicy")]  
0 references  
public async Task<IActionResult> EditRole(string id)
```

За простота коментираме изискването за Админ в началото на контролера, така че това да стане част само от EditRole action с неговия policy.

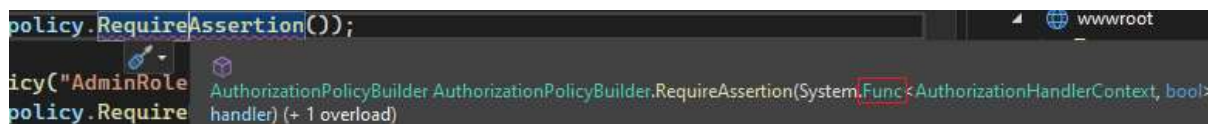
```
//[Authorize(Roles = "Admin")]  
3 references  
public class AdministrationController : Controller
```

Към момента дори не сме създали потребителска роля Super Admin, така че, когато впишем потребител, който е Admin и има права за 3-те: Create, Edit, Delete Role, той пак няма да види бутон Edit, а само Delete, т.к. не удовлетворява условието за Super Admin. Това демонстрира и въпросът от предходния урок: Edit е с най-висок приоритет, така че, ако има права за него, има права и върху Delete. Обратното не важи. Бутон Edit е скрит, а ако се опитаме да получим достъп директно от URL: <http://localhost:24077/Administration/EditRole/71ef2a67-38d4-42e5-88c0-2847ce6f6ca0> ни пренасочва



Създаваме Super Admin и потребител, който притежава всички права, така че да отговаря на всички изисквания Admin, Super Admin + Edit Role. Logout+Login, за да влязат в сила промените.

Сега имаме достъп, но това не е което искаме. Ние искаме потребителят да бъде Admin + Edit ИЛИ Super Admin, а не 3-те едновременно. За целта използваме персонализиран .RequireAssertion вместо .RequireClaim+.RequireRole.



Вижда се, че .RequireAssertion изисква Func тип параметър, който от своя страна прихваща параметър от тип AuthorizationHandlerContext и връща параметър тип Boolean.

Нека наречем параметъра context

```
options.AddPolicy("EditRolePolicy",
    policy => policy.RequireAssertion(context =>
        context.User.IsInRole("Admin") &&
        context.User.HasClaim(claim => claim.Type == "Edit Role" &&
            param.Prop.Method claim.Value == "true") ||
        context.User.IsInRole("Super Admin")
    ));
```

Parameter.Property.Method()

Остава да тестваме с 3 потребителя: единият Admin + Edit, другият само Super Admin, третият само Admin без Edit.

```
services.AddAuthorization(options =>
{
    options.AddPolicy("EditRolePolicy", policy => policy.RequireAssertion(context =>
        context.User.IsInRole("Admin") &&
        context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true") ||
        context.User.IsInRole("Super Admin")
    ));
});
```



Единственото интересно нещо в случая е, че Super Admin няма видимост към Manage бутона в топ менюто заради if условие в _Layout.

```
tml _Layout.cshtml X Startup.cs*
@if (signInManager.IsSignedIn(User) && User.IsInRole("Admin"))
{
    <li class="nav-item dropdown">
        <a class="nav-link dropdown-toggle text-light"
            href="#"
            id="navbarDropdownMenuLink"
            role="button"
            data-toggle="dropdown"
            @*data-hover="dropdown"*@
            aria-haspopup="true"
            aria-expanded="false">
            Manage
        </a>
    </li>
}
```

Нищо не пречи временно да закоментираме if условието, тогава Super Admin ще има видимост, ако не – директно от URL адреса има достъп до EditRole с видимост на Edit + Delete бутоните.

Вторият начин е да се състави функция, която връща bool дали условията са изпълнени:

Using a Func to Create a Custom Policy

```
services.AddAuthorization(options =>
{
    options.AddPolicy("EditRolePolicy", policy =>
        policy.RequireAssertion(context => AuthorizeAccess(context)));
});
```



```
private bool AuthorizeAccess(AuthorizationHandlerContext context)
{
    return context.User.IsInRole("Admin") &&
        context.User.HasClaim(claim => claim.Type == "Edit Role" && claim.Value == "true") ||
        context.User.IsInRole("Super Admin");
}
```

